Deliverable 6: Analytical Note

Student-to-Project Assignment Decision Support System

TECH5 Programming and Application Design, NHH Norwegian School of Economics

Group:	15
Team B (Engine):	Derek Liang, Aayush Mohan Kolhe, Ole Theodor Åmlid
Team A (Visualization):	Wilhelm Leikanger, Alexander Bangsund, Henrik Wassiluk
Date:	2 November 2026

Summary. We assign students to projects with an integer linear program (ILP). All business rules that must never be broken (one project per student, capacities, fixed assignments, mandatory skills) are hard constraints. Three strategies change only the objective: preference, skill fit, and a balanced strategy that first protects the worst-off student. Up to K distinct alternatives are generated with no-good cuts, and impossible problems are reported as infeasible rather than silently relaxed. Each alternative is evaluated on feasibility, efficiency, fairness and capacity, and recommendations are made per policy priority so that the decision-maker sees the trade-off instead of a single score.

1 Problem definition and modelling approach

1.1 Business context and decision dilemma

A programme coordinator must assign a cohort of students to semester-long business projects, as in the CEMS Business Projects at NHH. Students rank the projects they prefer and have skills at levels 1–5; projects have a minimum and maximum number of students, desired skills and some mandatory skills; and the administration may already have fixed some assignments. The objectives conflict: giving many students their first choice can leave a project without the skills it needs, and maximising average satisfaction can leave one student with a very poor outcome. Rezaeinia, Góez and Guajardo [1] study this setting for an NHH master's programme and show that efficiency and fairness have to be treated as separate criteria. Following that insight, our system does not hide the conflict behind one score: it produces several feasible alternatives that each favour a different criterion, and it reports indicators that make the differences visible.

1.2 Chosen methodology and justification

We use binary integer programming, modelled with Pyomo [3] and solved with the HiGHS solver [4] (`appsi_highs`), the same tools as in TECH4. The model is a generalised *assignment problem* [2, Ch. 9.3]: every student is assigned to exactly one project, but a project receives between l_p and u_p students instead of exactly one. Additional business rules are added with the standard binary-variable techniques for logical relationships [2, Ch. 12].

We chose ILP over the alternatives the brief mentions for four reasons.

1. **Every rule maps to one constraint.** Capacities, fixed assignments and mandatory skills are linear constraints, which makes the model easy to inspect and explain. A greedy rule-based procedure needs repair steps for minimum capacities and mandatory skills, and stable matching loses its guarantees once lower quotas are added.
2. **Infeasibility is proven.** When no assignment satisfies the hard constraints, branch-and-bound [2, Ch. 12.6] proves it. A heuristic that fails cannot distinguish "impossible" from

"not found".

3. **Distinct alternatives are exact.** No-good cuts (constraint (8)) guarantee that every alternative differs from all earlier ones in at least one student's project, which is exactly the brief's definition of a distinct alternative.
4. **The problem is small.** The model has $|S| \cdot |P| + |P|$ binary variables: 44 for the 10-student demo cohort, which HiGHS solves in a few hundredths of a second inside the single-process Flask application.

2 Mathematical formulation

2.1 Index sets

- S: students in the cohort, $|S| = n$.
- P: projects, $|P| = m$.
- K_p: skills required by project p (mandatory and desired); $M_p \subseteq K_p$: the mandatory ones.
- $F \subseteq S \times P$: fixed administrative assignments; $S_F = \{s : (s, p) \in F\}$: students with a fixed assignment.
- $\mathcal{A} = \{A^{(1)}, \ldots, A^{(j)}\}$: solutions found earlier in the same run, each a set of n student–project pairs.

2.2 Parameters

- $l_p \in \mathbb{Z}_{\geq 0}$, $u_p \in \mathbb{Z}_{\geq 1}$: minimum and maximum number of students on project p, $l_p \leq u_p$.
- r_{sp}: preference rank of student s for project p (1 = first choice). An unranked project gets $r_{sp} = m + 1$.
- $u_{sp} = (m + 1 - r_{sp})/m \in [0, 1]$: preference score, 1 for a first choice and 0 for an unranked project.
- $\lambda_{sk} \in \{0, \ldots, 5\}$: level of student s in skill k (0 if not held); $L_{pk} \in \{1, \ldots, 5\}$: level project p requires in skill k.
- $\sigma_{sp} \in [0, 1]$: skill fit of student s on project p, the average share of each required level that the student reaches:

$$\sigma_{sp} = \begin{cases} \dfrac{1}{|K_p|} \displaystyle\sum_{k \in K_p} \min\left(1, \frac{\lambda_{sk}}{L_{pk}}\right) & \text{if } K_p \neq \emptyset, \\ 1 & \text{if } K_p = \emptyset. \end{cases} \tag{1}$$

  A project without requirements is a neutral, perfect fit. A student with no relevant skills has fit 0 but can still be assigned.

- $\varepsilon = 0.001$: tie-break weight (Section 2.5).

2.3 Decision variables

$$x_{sp} = \begin{cases} 1 & \text{if student } s \text{ is assigned to project } p, \\ 0 & \text{otherwise,} \end{cases} \qquad \forall s \in S,\ p \in P,$$

$$y_p = \begin{cases} 1 & \text{if project } p \text{ runs (has at least one student),} \\ 0 & \text{otherwise,} \end{cases} \qquad \forall p \in P.$$

The balanced strategy adds one continuous auxiliary variable $z \geq 0$, the worst assigned rank (Section 2.5).

2.4 Hard constraints

These constraints are identical for every strategy; they are implemented once, in the class `AssignmentModel` in `app/engine/solvers.py`.

$$\sum_{p \in P} x_{sp} = 1 \qquad \forall s \in S \tag{2}$$

$$\sum_{s \in S} x_{sp} \geq l_p \qquad \forall p \in P \tag{3}$$

$$\sum_{s \in S} x_{sp} \leq u_p\, y_p \qquad \forall p \in P \tag{4}$$

$$\sum_{s \in S} x_{sp} \geq y_p \qquad \forall p \in P \tag{5}$$

$$x_{sp} = 1 \qquad \forall (s,p) \in F \tag{6}$$

$$\sum_{s \in S\,:\,\lambda_{sk} \geq L_{pk}} x_{sp} \geq y_p \qquad \forall p \in P,\ k \in M_p \tag{7}$$

$$\sum_{(s,p) \in A} x_{sp} \leq n - 1 \qquad \forall A \in \mathcal{A} \tag{8}$$

$$x_{sp},\ y_p \in \{0,1\} \qquad \forall s \in S,\ p \in P$$

(2) **One project per student.** Every student receives exactly one project.

(3)–(4) **Capacity.** Each project receives between l_p and u_p students. The minimum applies to every project, consistent with input validation.

(4)–(5) **Linking y_p to x_{sp}.** Together these force $y_p = 1$ exactly when project p has at least one student. Constraint (4) is the big-M linking constraint of the fixed-charge problem [2, Ch. 12.3] with the tightest valid $M = u_p$; constraint (5) is a contingent decision ("y_p only if some x_{sp}").

(6) **Fixed assignments.** Administrative locks hold in every solution and every alternative.

(7) **Mandatory skills.** A running project must include at least one student who meets each mandatory level. This is a set-covering constraint [2, Ch. 12]: the qualified students "cover" the requirement. Multiplying by y_p means that a project with $l_p = 0$ may stay empty instead of requiring an expert. We interpret "mandatory" at team level (at least one qualified member); the stricter reading, every member qualified, would instead set $x_{sp} = 0$ for every unqualified student.

(8) **No-good cuts.** If every student kept the project they had in an earlier solution A, the left-hand side would equal n. Requiring at most $n - 1$ forces at least one student to move, so each new solution is distinct from all earlier ones. The constraint has the same structure as subtour elimination in the travelling salesman problem: it removes unwanted integer solutions and leaves all others feasible.

2.5 Objectives: three strategies

Each strategy is a subclass of `AssignmentStrategy` (Strategy pattern) and differs only in its objective.

Strategy 1: preference-oriented (`PREFERENCE`). Minimise the total assigned rank; skill fit only breaks ties:

$$\max \; Z_{\text{pref}} = \sum_{s \in S} \sum_{p \in P} \left(-r_{sp} + \varepsilon \, \sigma_{sp} \right) x_{sp}. \tag{9}$$

Total rank changes in integer steps, while the tie-break term is at most $\varepsilon n = 0.01$ for the demo cohort, so it can never outweigh one rank step as long as $n < 1/\varepsilon = 1000$.

Strategy 2: skill-fit-oriented (`SKILL_FIT`). Maximise total skill fit; preferences only break ties:

$$\max \; Z_{\text{skill}} = \sum_{s \in S} \sum_{p \in P} \left(\sigma_{sp} + \varepsilon \, u_{sp} \right) x_{sp}. \tag{10}$$

Strategy 3: balanced trade-off (`BALANCED`). A two-stage (lexicographic) objective. Stage 1 is a minimax problem, like the districting problem in which the largest deviation is made as small as possible: it finds the smallest achievable worst rank z^* among students without a fixed assignment,

$$z^* = \min \; z \quad \text{s.t.} \quad z \geq r_{sp} \, x_{sp} \;\; \forall s \in S \setminus S_F, \; p \in P, \quad (2)\text{–}(8). \tag{11}$$

Fixed students are excluded because their rank is not a choice the model can improve. Stage 2 keeps that guarantee and maximises an equal blend of preference score and skill fit:

$$\max \; Z_{\text{bal}} = \sum_{s \in S} \sum_{p \in P} \left(\tfrac{1}{2} \, u_{sp} + \tfrac{1}{2} \, \sigma_{sp} \right) x_{sp} \quad \text{s.t.} \quad x_{sp} = 0 \;\; \forall s \notin S_F, \; p : r_{sp} > z^*, \quad (2)\text{–}(8). \tag{12}$$

2.6 Generating distinct alternatives

`AssignmentEngine.generate_alternatives` produces up to K alternatives (default $K = 3$):

1. Alternative 1 uses the strategy the user selected; alternatives 2 and 3 use the two other strategies, so each alternative favours a different criterion.
2. Each solve includes a no-good cut (8) for every earlier solution, so all alternatives are pairwise distinct.
3. The constraints are the same for every strategy, so if one solve is infeasible, no further distinct assignment exists. Generation then stops and reports the number found (`distinct_alternatives`). It never fabricates duplicates.
4. If the first solve is infeasible, the run is reported as `INFEASIBLE` with no solutions.

3 Evaluation indicators

Let $p(s)$ be the project assigned to student s in a solution, and $\tilde{r}_s = r_{s\,p(s)}$ its rank (with $m + 1$ for an unranked project). All indicators are computed from the solution itself, independently of the solver.

3.1 Feasibility

The solution is re-checked against every hard constraint, so that a solver or data error cannot pass unnoticed:

- completeness: $\sum_{s,p} x_{sp} = n$, each student exactly once;
- capacity compliance: $l_p \leq \sum_s x_{sp} \leq u_p$ for every p;
- fixed-assignment adherence: $x_{sp} = 1$ for every $(s, p) \in F$;
- mandatory coverage: every running project has at least one qualified student for each $k \in M_p$.

Any violation is listed by name and marks the solution as infeasible.

3.2 Efficiency

- **Mean skill fit**, our main efficiency measure: $\bar{\sigma} = \frac{1}{n} \sum_{s \in S} \sigma_{s\,p(s)} \in [0, 1]$. It answers "how well, on average, do students meet the levels their projects ask for?". 1 means every requirement is fully met.
- **Unmet desired requirements**: $U_{\text{soft}} = \left|\{(s,k) : k \in K_{p(s)} \setminus M_{p(s)},\ \lambda_{sk} < L_{p(s)k}\}\right|$, the number of desired (non-mandatory) skill gaps, which shows *where* skill fit is lost.
- **Mean preference rank**: $\bar{r} = \frac{1}{n} \sum_{s \in S} \tilde{r}_s$, where lower is better.

3.3 Fairness

Aggregate performance is not enough, so we report how outcomes are distributed:

1. **Rank distribution**: $c_k = |\{s : \tilde{r}_s = k\}|$ for $k = 1, \ldots, m$, plus c_{unranked}.
2. **Students outside their preferences**: $U = |\{s : p(s) \text{ not ranked by } s\}|$.
3. **Worst individual outcome**: $r_{\max} = \max_{s \in S} \tilde{r}_s$.
4. **Our proposed indicator: the Gini coefficient of ranks**,

$$G = \frac{\sum_{i \in S} \sum_{j \in S} |\tilde{r}_i - \tilde{r}_j|}{2\, n \sum_{i \in S} \tilde{r}_i} \in [0, 1). \tag{13}$$

Justification. $\bar{r}$ and $r_{\max}$ do not show whether disappointment is spread evenly or concentrated on a few students. G does: it is 0 when every student receives the same rank, and grows as outcomes become more unequal. For ranks $(1, 1, 1, 4)$, the mean rank is 1.75 and $G = 0.32$; for $(2, 2, 2, 2)$ the mean rank is worse, 2.00, but $G = 0$. Showing both lets the decision-maker see the efficiency–equity trade-off directly. *Interpretation.* Because a low rank is good, a high G means unequal *disappointment*, not unequal wealth. G measures spread, not level: ranks $(3, 3, 3, 3)$ also give $G = 0$, so it is always reported together with $\bar{r}$ and $r_{\max}$. The rank variance $\frac{1}{n} \sum_s (\tilde{r}_s - \bar{r})^2$ is reported as a supporting measure.

How the interface presents fairness. The brief asks that the application should not simply display these numbers but help the user understand what they imply. On the solution page, the four indicators above are therefore shown twice: once as plain figures in the summary cards, and once in a fairness section further down that says which students are behind the numbers.

- The rank distribution c_k is drawn as a horizontal bar per rank, from first choice down to "not ranked", so the spread can be seen without reading the numbers. A tall first bar and no red bar is the ideal.
- The U students assigned outside their preferences are listed by name, with the project they received and the projects they had ranked. A count alone does not tell the coordinator who to talk to.
- The worst individual outcome $r_{\max}$ is shown as the student (or students, if several tie) who received it, together with their first choice. If anyone is unranked, that group is treated as the worst case regardless of $r_{\max}$.
- The Gini coefficient is given with a colour scale and one of four readings: *very even* ($G < 0.10$), *fairly even* ($0.10 \le G < 0.25$), *uneven* ($0.25 \le G < 0.40$) and *very uneven* ($G \ge 0.40$). The thresholds are our own judgement, chosen so that the demo cohorts land where we felt they should: $G = 0.08$ with one student on a second choice reads as "very even", and the four-student example in Section 4 with $G = 0.32$ reads as "uneven". The bands are a reading aid, not a standard, and the exact value is always shown next to them.

Above the section, one sentence sums up the solution, for example "9 of 10 students received their first or second choice. The least satisfied student received their 3rd choice." It is built from $c_1 + c_2$, U and $r_{\max}$, and names the unranked students when there are any.

3.4 Capacity utilisation

$$\text{Util}_p = \frac{\sum_s x_{sp}}{u_p} \times 100\%, \qquad \text{Util} = \frac{n}{\sum_p u_p} \times 100\%.$$

Projects at their minimum or maximum are flagged, since they leave no slack when the situation changes.

4 Trade-off analysis and solution comparison

4.1 Comparative deltas

For a baseline solution A and an alternative B, each indicator is reported as a difference $B - A$:

$$\Delta\bar{\sigma} = \bar{\sigma}_B - \bar{\sigma}_A, \quad \Delta\bar{r} = \bar{r}_B - \bar{r}_A, \quad \Delta r_{\max} = r_{\max,B} - r_{\max,A}, \quad \Delta G = G_B - G_A.$$

A positive $\Delta\bar{\sigma}$ is an improvement; positive $\Delta\bar{r}$, $\Delta r_{\max}$ and ΔG are deteriorations.

4.2 Who is affected

- Changed assignments: $C = \{(s, p_A(s), p_B(s)) : p_A(s) \neq p_B(s)\}$; $|C| \ge 1$ for distinct alternatives.
- Worsened students: $\tilde{r}_s^B > \tilde{r}_s^A$; improved students: $\tilde{r}_s^B < \tilde{r}_s^A$.

This answers the brief's questions: what improves, what becomes worse, and who is affected.

4.3 Dominance

Alternative B is *dominated* by A if A is at least as good on mean rank and mean skill fit and strictly better on at least one ($\bar{r}_A \le \bar{r}_B$, $\bar{\sigma}_A \ge \bar{\sigma}_B$, not both equal). A dominated alternative offers

no real trade-off. Each strategy's optimum is Pareto-optimal for its own criterion, but no-good cuts only guarantee distinctness, so a later alternative can be dominated. Dominated alternatives are therefore flagged: the engine adds a warning and the numbers of the dominating alternatives (`diagnostics["dominated_by"]`) to the solution, and the comparison summary states that there is no real trade-off.

4.4 How the interface presents trade-offs

Two pages show these quantities to the user.

Compare page. The user picks any two solutions, grouped by run, and sees the deltas of Section 4.1 side by side, the counts of students whose rank improved and worsened, and a table of every changed assignment with the student's name, both projects and the rank change. Runs without solutions are left out of the selection, so an infeasible run cannot be chosen by mistake.

Recommendations page. For each policy priority the page shows the recommended alternative, the criteria and evidence from Section 5, and then answers the brief's three questions against every other non-dominated alternative:

- **What improves and what gets worse** is written out in words from the deltas of five indicators: mean skill fit, mean rank, worst rank, students outside their preferences and unmet desired requirements. Only indicators that actually change are listed, and if none change the page says so.
- **Who is affected** is a table of the students who move, each with a before-and-after label such as "1st choice → 3rd choice" and a count of how many are better and worse off.
- **Dominated alternatives** are excluded from these comparisons, because comparing against an option that is worse on every count would be misleading. Instead a banner states which alternative is dominated, by which, and why.

When a run has exactly one feasible solution, all three priorities would pick it, so the page shows one recommendation rather than three identical ones.

4.5 Illustrative results

Table 1: Standard demo cohort: 10 students, 4 projects with capacity 1–3 (44 binary variables, each solve < 0.1 s).

Alternative	$\bar{r}$	$r_{\max}$	c_1	U	G	$\bar{\sigma}$	U_{soft}	Moved
1 `PREFERENCE`	1.00	1	10	0	0.000	0.983	1	–
2 `SKILL_FIT`	1.10	2	9	0	0.082	1.000	0	1 vs. alt. 1
3 `BALANCED`	1.10	2	9	0	0.082	0.983	1	2 vs. alt. 2

Table 1 shows a clean trade-off between alternatives 1 and 2: moving a single student (S06) to their second choice raises mean skill fit from 0.983 to 1.000 and removes the only unmet desired requirement. Alternative 3 is dominated by both: it has a worse mean rank than alternative 1 at the same skill fit, and the same mean rank as alternative 2 at a lower skill fit. The engine therefore attaches the warning "Dominated by alternatives 1 and 2". Because alternative 1

Table 2: Fixed-assignment scenario: S02 is locked to P2, their third choice.

Alternative	$\bar{r}$	$r_{\max}$	$c_1/c_2/c_3$	G	$\bar{\sigma}$	U_{soft}
1 `PREFERENCE`	1.20	3	9/0/1	0.150	0.967	2
2 `SKILL_FIT`	1.30	3	8/1/1	0.192	0.983	1
3 `BALANCED`	1.30	3	8/1/1	0.192	0.967	2

already gives everyone a first choice, the balanced strategy cannot improve the worst rank in this cohort; it only becomes a different alternative because of the no-good cuts.

In Table 2 the lock holds in every alternative, and its cost is visible: compared with Table 1, mean rank rises from 1.00 to 1.20 and the worst rank is 3, caused entirely by the fixed student.

The demo cohort is too easy to show the balanced strategy at work. In a 4-student, 4-project example with one seat per project (preferences S1, S2: D,C,A,B; S3: A,C,B,D; S4: C,A,D,B), `PREFERENCE` reaches the lowest total rank, 7 ($\bar{r} = 1.75$), by giving S2 their fourth choice ($r_{\max} = 4$, $G = 0.32$). `BALANCED` accepts a higher total of 8 ($\bar{r} = 2.00$) to guarantee everyone their third choice or better ($r_{\max} = 3$, $G = 0.19$). This is precisely the trade-off the brief describes: protecting the worst-off student reduces average quality.

5 Transparent recommendation rules

Recommendations are generated in `app/engine/recommendations.py` for each policy priority separately. Each recommendation lists the criteria used, the calculated evidence (indicator values) and the advice, so that calculated facts are kept apart from normative judgement.

Criteria are applied lexicographically: the second criterion only decides between alternatives tied on the first.

5.1 Policy priority 1: student preference maximisation

Criterion: minimise U, then $\bar{r}$, then $r_{\max}$. *Advice:* adopt when student satisfaction and few appeals matter most. *Trade-off warning:* projects may get weaker skill coverage.

5.2 Policy priority 2: project delivery and technical competence

Criterion: maximise $\bar{\sigma}$, then minimise U_{soft}. *Advice:* adopt when partners demand strong technical capability. *Trade-off warning:* some students receive lower-ranked projects.

5.3 Policy priority 3: balanced and egalitarian compromise

Criterion: minimise G, then rank variance, then $\bar{r}$. *Advice:* adopt to avoid concentrating disappointment on a few students. *Trade-off warning:* average satisfaction may be lower.

When several alternatives tie on all criteria, the one with the lowest alternative number is recommended. The same alternative can be recommended under more than one priority; this is shown rather than hidden, because it tells the decision-maker that the priorities do not conflict for this cohort.

6 Assumptions, boundary conditions and edge cases

Assumptions.

1. **Unranked projects** are allowed but cost rank $m + 1$, worse than any ranked project, and are counted in U.
2. **Ranks are evenly spaced**: the step from first to second choice weighs the same as from second to third.
3. **Skill levels** are whole numbers from 1 to 5, for students and requirements alike; other values are rejected. A skill missing from a profile counts as level 0. A project with no requirements has fit 1.
4. **Preferences**: each student is expected to rank at least three projects, each with a distinct rank. Fewer gives a warning, not an error.
5. **Mandatory skills** are met at team level (at least one qualified member). This is our documented interpretation of the brief; the stricter reading would set $x_{sp} = 0$ for every unqualified student, a one-line change in the model.
6. **Ties** between students with identical preferences are broken by the secondary goal (ε term) and then by the solver; the result is a valid optimum, not a judgement about which student deserves the seat.

Handling of edge cases. Every case in the brief has a defined outcome; none may end in an unexplained crash. Input validation (`app/domain/validation.py`) runs before the solver.

Feasibility boundaries. The problem is provably infeasible if $\sum_p u_p < n$, if $\sum_p l_p > n$, if any project has more fixed students than u_p, or if a mandatory level is held by no student in the cohort. Validation detects these directly. Validation is deliberately conservative: it treats the last case as infeasible even when the project has $l_p = 0$ and could stay empty under constraint (7). All other infeasibility, which arises from combinations of constraints, is proven by the solver. Data errors are reported before infeasibility: an empty cohort, for example, also violates $\sum_p l_p \leq n$, but the run is `INVALID_INPUT`, because the input must be corrected before feasibility means anything.

7 Limitations

1. **Computational complexity.** Integer programming is NP-hard in general, and the number of binary variables grows as $n \cdot m$. At realistic cohort sizes the model remains small, but we have only measured solve times on the demo cohorts.
2. **Linear preferences.** Evenly spaced ranks assume that a student is as disappointed moving from first to second choice as from second to third, which need not hold.
3. **Individual, not team, skill fit.** σ_{sp} is measured per student. The model does not capture complementary skills, team diversity or personal compatibility.
4. **Weights and tie-breaks are judgements.** The $\frac{1}{2}/\frac{1}{2}$ blend in `BALANCED` is a choice, not a fact. In `SKILL_FIT` the tie-break term can outweigh very small differences in total skill fit (below εn), and stage 2 of `BALANCED` has no tie-break, so the solver chooses between equally good assignments. We have not run a sensitivity analysis on these parameters [2, Ch. 7].

Table 3: Edge cases and the system's response.

Case	Outcome	Behaviour
Empty cohort or catalogue; duplicate or unknown identifiers; invalid capacities ($u_p \leq 0$, $l_p < 0$, $l_p > u_p$)	Reject	Named validation error; the solver is not run.
Contradictory fixed assignments (one student fixed twice)	Reject	Named validation error.
Fewer than three ranked projects, or none	Warn	Run proceeds; unranked projects cost $m + 1$.
Tied ranks (one rank used twice); the same project ranked twice	Reject	`INVALID_INPUT`; the ranking is ambiguous.
Skill or requirement level outside 1–5	Reject	`INVALID_INPUT`, naming the student or project.
Uncatalogued skill in a profile or requirement	Warn	Run proceeds; the skill counts only where defined.
Missing skills; project without requirements	Accept	Fit 0 or 1 respectively (equation (1)).
$\sum_p u_p < n$; $\sum_p l_p > n$; more fixed students than u_p; a mandatory skill no student has	Infeasible	Proven before solving, with the reason stated.
Infeasibility arising only from combinations (e.g. two projects each need the single qualified student)	Infeasible	Proven by the solver; status `INFEASIBLE`, no solutions.
Only one (or fewer than K) feasible solutions	Partial	The solutions found are returned; the stop reason states that no further distinct solution exists.
Duplicate alternatives	Prevented	Impossible by construction (no-good cuts).
Equal aggregate scores	Accept	Tie-break term; in recommendations the lowest alternative number wins.
Solver time limit (15 s) or database failure	Fail	Run status `FAILED`; the solver status (e.g. `NOT_SOLVED`) is kept in the diagnostics.

5. **Alternatives are distinct, not necessarily efficient.** No-good cuts guarantee that alternatives differ, not that they are Pareto-optimal. Dominated alternatives are flagged (Table 1) but still generated.

6. **Runs are tied to the current cohort.** Stored assignments reference students and projects with `ON DELETE CASCADE`. Loading a demo scenario therefore deletes all earlier runs, so they are not left with indicators but no allocations. Deleting a single student or project by hand still removes that student's or project's rows from earlier solutions. Keeping a full history across cohorts would require storing a snapshot of the input with each run, which changes the shared schema.

7. **Static, self-reported data.** The model assumes the input data are correct and fixed. Self-reported skill levels may be inaccurate, and mid-semester changes require a new run.

8. **Plain-language readings are judgements.** The four Gini bands and the one-sentence headline on the solution page compress the indicators into words. The thresholds were set by eye on the demo cohorts, and a headline that counts first and second choices says nothing about third choices. Both are meant to orient the reader, who still has to look at the figures and the named students below them.

9. **What-if analysis is limited to locks.** The only way to explore a change from the results pages is "lock and rerun", which fixes one student and solves again. There is no way to try a temporary change, compare it, and discard it; every experiment becomes a saved fixed assignment that must be removed by hand afterwards. On the data-entry side, a form that fails validation is returned empty, so the user retypes everything. These are usability gaps rather than analytical ones, but they slow down exactly the iterative use the brief describes.

References

[1] N. Rezaeinia, J. C. Góez and M. Guajardo. Efficiency and fairness criteria in the assignment of students to projects. *Annals of Operations Research*, 319(2):1717–1735, 2022. `https://doi.org/10.1007/s10479-021-04001-7`

[2] F. S. Hillier and G. J. Lieberman. *Introduction to Operations Research*. McGraw-Hill. Chapters 7 (sensitivity analysis), 9.3 (the assignment problem) and 12 (integer programming), as covered in TECH4 Mathematics for Decisions, NHH, autumn 2025.

[3] M. L. Bynum, G. A. Hackebeil, W. E. Hart, C. D. Laird, B. L. Nicholson, J. D. Siirola, J.-P. Watson and D. L. Woodruff. *Pyomo: Optimization Modeling in Python*, 3rd edition. Springer, 2021.

[4] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.